BZ 313 Yazılım Mühendisliği

Hafta 2

Modern Yazılım Geliştirme Süreçleri ve Araç Kurulumu

Erciyes Üniversitesi
Bilgisayar Mühendisliği Bölümü

2026–2027 Güz

Bu haftanın hedefleri

Bu dersin sonunda şunları yapabileceksiniz:

- Bir yazılım sürecinin ne işe yaradığını ve neden tek bir doğru sürecin olmadığını açıklayabileceksiniz.
- Şelale, yinelemeli ve çevik yaklaşımları projenin özelliklerine göre karşılaştırıp seçim yapabileceksiniz.
- Scrum ritmini ve Kanban akış sınırlarını dönem projenize uygulayabileceksiniz.
- Git ile dal açıp birleştirme isteği süreciyle çalışabilecek, çakışma çözebileceksiniz.
- Grubunuzun deposunu, dal korumasını ve Kanban panosunu kurabileceksiniz.

Bugünün akışı

- Açılış, hedefler, bugünün akışı
- Grup Dakikası
- Bölüm 1: Süreç nedir, şelale ve yinelemeli geliştirme
- Uygulamalı örnek ve alıştırma: hangi projeye hangi süreç?
- Bölüm 2: Çevik yaklaşım, Scrum, Kanban, DevOps
- Uygulamalı örnek ve alıştırma: sprint planı ve pano
- Bölüm 3: Git, GitHub, depo kurulumu, lisans
- Alıştırma: depo geçmişini denetlemek
- Bu hafta projenizde, kaynaklar, kapanış

Grup Dakikası

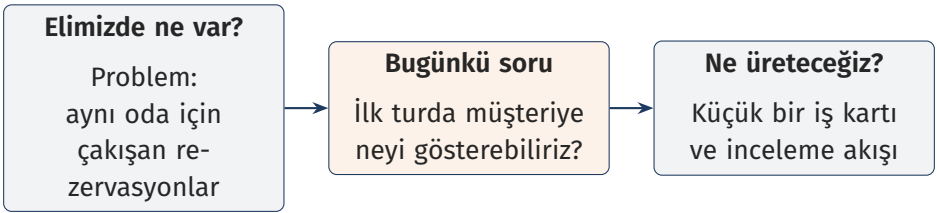
Kurallar

- Her gruba bir slayt ve bir dakika ayrılır.
- Konuşmacı her hafta değişir.
- Slayt dersten önce yüklenir.
- Süre bitince söz sonraki gruba geçer.

Bu haftanın sorusu

Grubunuz kim, projeniz ne, müşteriniz kim?

Bir fikri küçük bir çalışan dilime dönüştürelim



Önce düşünelim

Önce bütün ekranları çizmek mi, tek oda için çalışan rezervasyon akışını göstermek mi daha erken geri bildirim sağlar?

Ortak örnek: kütüphane çalışma odası rezervasyonu. Sayılar ve kurallar öğretim amaçlı varsayımlardır.

Bölüm 1: Süreç, şelale ve yinelemeli geliştirme

Süreç: işi kim, ne zaman, hangi sırayla yapar?

Yazılım süreci

Bir yazılımı fikirden çalışan ve bakımı yapılabilir bir ürüne dönüştüren etkinlikler, bunların sıralaması ve çıktılarının tanımıdır.

- Süreç, hangi işin yapılacağını değil, işin *nasıl örgütlendiğini* belirler.
- Tek kişilik ödevde süreç görünmezdir; dokuz kişilik grupta her şeydir.
- İyi süreç üç soruyu her an yanıtlar: Ne durumdayız? Sırada ne var? Ne engelliyor?

Süreç olmadan dokuz kişi ne yapar?

Gözlenen davranış

- İki kişi aynı ekranı ayrı ayrı yazar.
- Kimse veri tabanı şemasına karar vermez.
- Son hafta tek kişi dört gün uykusuz kod yazar.
- Müşteri ürünü ilk kez teslimde görür.

Sürecin verdiği yanıt

- Görev panosu: kim neyi almış, görünür.
- Karar kaydı: kararı yazılı tutar.
- Zaman kutusu: çalışma belirli aralıklarla planlanır.
- Haftalık görüşme: geri bildirim erken gelir.

Süreç bürokrasi değildir. Bürokrasi, kimsenin okumadığı belgedir.

Sürecin dört temel bileşeni

- **Etkinlikler:** ne yapılır? Gereksinim toplama, tasarım, kodlama, test.
- **Sıra ve tekrar:** hangi sırayla, kaç kez? Bir kez mi, her iki haftada bir mi?
- **Roller:** kim karar verir, kim uygular, kim onaylar?
- **Çıktılar:** her etkinlik sonunda elde somut olarak ne var?

⚠ Çıktısı olmayan etkinlik

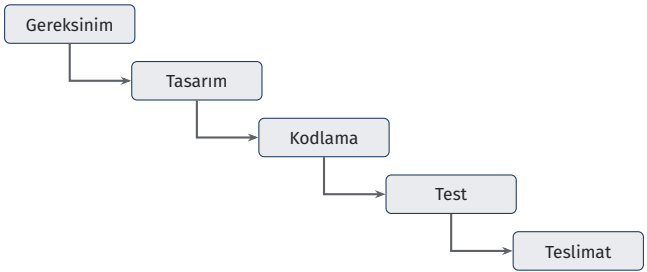
"Tasarımı konuştuk" bir çıktı değildir. Çıktı, bir diyagram, bir karar kaydı veya bir iş kaydıdır. Yazılı olmayan karar zamanla farklı yorumlanır.

Ağır ve hafif süreçler

	Ağır süreç	**Hafif süreç**
Belge	Ayrıntılı, önden onaylı	Az, çalışan ürünle birlikte
Değişiklik	Resmî talep ve onay	Sonraki döngüde ele alınır
Planlama ufku	Aylar	Bir iki hafta
Uygun olduğu yer	Sözleşmeli, regüle, güvenlik kritik	Belirsiz, hızlı değişen alan
Riski	Yanlış şeyi mükemmel yapmak	Yön kaybı ve teknik borç

- Aynı e-ticaret sitesi ağır süreçle 17 ayda, hafif süreçle 3 ayda ilk sürümü verebilir.
- Fark yalnızca hız değil, *ne zaman karar verdiğinizdir*. Ağır süreç erken, hafif süreç geç bağlanır.

Şelale modeli



- Her aşama bir öncekinin onaylı çıktısıyla başlar; geri dönüş pahalıdır.
- Gereksinimler gerçekten sabitse ve alan iyi biliniyorsa işe yarar.
- Saf hâliyle nadiren uygulanır. Uygulanan biçim *değiştirilmiş şelaledir*: aşamalar örtüşür ve geri bildirim döngüleri bulunur.

Şelale neden hâlâ yaşıyor?

- **Sözleşme:** kamu ihalesi “ne teslim edileceğini” önden yazılı ister.
- **Denetim:** tıbbi cihaz, havacılık ve finans yazılımında izlenebilir belge zorunludur.
- **Donanım bağımlılığı:** üretim hattı bir kez kurulur, sonradan değişmez.
- **Dış bağımlılık:** entegre olacağınız kurum arayüzünü yıl başında dondurur.

⚠ Yaygın yanlış

“Şelale kötüdür, çevik iyidir” cümlesi bir süreç seçimi değildir. Doğru soru şudur: gereksinimlerim ne kadar belirsiz ve geri bildirimi ne kadar hızlı alabilirim?

Yinelemeli iyileştirme

Yineleme

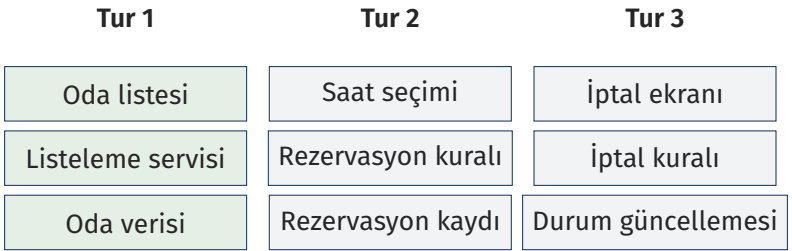
Sistemin çalışan ama eksik bir sürümünü üretip, değerlendirip, bir sonraki turda genişletme döngüsüdür.

- Her tur gereksinim, tasarım, kod ve testin küçük bir dilimini içerir.
- İlk turda “ince ama uçtan uca” bir dilim seçilir: giriş yap, tek ürünü listele.
- Böylece mimari riski dönemin başında görürsünüz, sonunda değil.

Yemekhane siparişi ilk turu

Menü listesi ekranı ve sabit veriyle çalışan bir servis. Ödeme, bildirim, yönetim paneli yok. Müşteri bunu ikinci haftanın sonunda görür ve “menü kategoriye göre gruplanmalı” der.

Dilimleme: dikey mi, yatay mı?



- **Sütun tamamlamak:** ekran + servis + veri; gösterilebilir bir iş.
- **Yalnızca satır tamamlamak:** veri hazırdır; kullanıcı henüz işini yapamaz.

Sorun: İlk tur sonunda müşteri hangi işi gerçekten deneyebilir?

Süreç seçim ölçütleri

Soru	Ağıra yakın	Hafife yakın
Gereksinimler ne kadar belirsiz?	Az	Çok
Hatanın bedeli nedir?	Can, para, ceza	Kullanıcı rahatsızlığı
Geri bildirimi ne kadar hızlı alabilirim?	Aylarda	Günlerde
Denetim veya sözleşme zorluyor mu?	Evet	Hayır
Ekip alanı ne kadar tanıyor?	İyi tanıyor	Yeni öğreniyor

Bu hafta projenizde

Dönem projenizde tabloyu doldurun. Çoğu grup için sonuç şudur: gereksinimler belirsiz, hata bedeli düşük, geri bildirim haftalık. Bu nedenle az belgeyle, iki haftalık yinelemelerle çalışın.

Alıştırma 1

Alıştırma (10 dk — gruplarınızla)

Aşağıdaki iki proje için önceki slayttaki beş ölçütü doldurun ve bir süreç önerin. Önerinizi tek cümlelik bir gerekçeyle yazın.

1. Bir üniversitenin **öğrenci kayıt** sistemini sıfırdan yazmak. Mevzuat sabit, yılda bir kez kullanılıyor, hata öğrencinin kaydını bozuyor.
2. Bir kampüs kulübü için **etkinlik duyuru** uygulaması. Kimse ne isteyeceğini bilmiyor, kullanıcılar öğrenciler, hata bedeli düşük.

Alıştırma 1: çözüm

Çözüm

1. Öğrenci kayıt sistemi: belirsizlik düşük (mevzuat), hata bedeli yüksek (öğrencinin kaydı ve mezuniyeti), geri bildirim yavaş (yılda bir kayıt dönemi), denetim var, alan iyi tanınıyor. ⇒ *Ağıra yakın*: gereksinimler yazılı ve onaylı, kapsamlı test, kademeli geçiş. Ancak arayüz tarafı yine prototiple kullanıcıya doğrulanmalı.

2. Etkinlik duyuru uygulaması: belirsizlik yüksek, hata bedeli düşük, geri bildirim çok hızlı (öğrenciler hemen kullanır), denetim yok. ⇒ *Hafif*: iki haftalık döngüler, her döngüde canlıya al, kullanım verisine bak.

- Aynı kurum içinde bile projeye göre farklı süreç seçilir.
- "Şirketimizin süreci budur" cümlesi, bu tabloyu doldurmamış olmanın işaretidir.

Bölüm 2: Çevik yaklaşım, Scrum, Kanban, DevOps

Çevik Manifesto'nun dört değeri

Bir tarafı diğerine *tercih ederiz*; sağdakini değersiz saymayız.

Tercih edilen		**Buna karşı**
Bireyler ve etkileşim	>	Süreçler ve araçlar
Çalışan yazılım	>	Kapsamlı belge
Müşteriyle iş birliği	>	Sözleşme pazarlığı
Değişime uyum	>	Planı izlemek

- 2001'de on yedi yazılımcının ortak metni. Bir yöntem değil, bir değerler kümesidir.
- "Belge yazmayacağız" demez. "Çalışmayan yazılımın belgesi işe yaramaz" der.

Çeviklik ne değildir?

- Plansızlık değildir. Planlama daha sık, daha kısa vadeli yapılır.
- Belgesizlik değildir. Belge, işe yarayan kadar ve zamanında yazılır.
- Toplantı bolluğu değildir. Toplantı kısadır, çıktısı vardır.
- “Müşteri ne derse yaparız” değildir. Kapsam pazarlığı yine yapılır.

⚠ Sahte çeviklik

Günlük toplantı yapılır ama pano güncellenmez; sprint adı konur ama kapsam sprint içinde büyür; retrospektif yapılır ama hiçbir şey değişmez. Toplantıların adını kullanmak yetmez; pano ve çalışma biçimi de buna göre değişmelidir.

Scrum: roller

Ürün sahibi

- İş birikiminin önceliğine karar verir.
- Müşterinin sesidir.
- “Ne?” sorusunun sahibi.

Geliştirme ekibi

- İşi kendi kendine örgütler.
- Nasıl yapılacağına karar verir.
- Kaliteden sorumludur.

Scrum yöneticisi

- Engelleri kaldırır.
- Süreci korur.
- Patron değildir.

Bu hafta projenizde

Dönem projenizde ürün sahibi rolünü *iletişim sorumlusu*, Scrum yöneticisi rolünü *koordinatör* taşır. Her iki hafta rol değişir.

Scrum: sprint ve etkinlikleri

Etkinlik	Amacı	Süre
Sprint planlama	Bu turda hangi işler bitecek, hedef ne?	1–2 saat
Günlük toplantı	Dün ne bitti, bugün ne var, engel var mı?	10–15 dk
Sprint gözden geçirme	Çalışanı müşteriye göster, geri bildirim al	30–60 dk
Retrospektif	Süreci nasıl iyileştiririz?	30 dk

- Sprint uzunluğu sabittir (bu derste iki hafta) ve uzatılmaz.
- Her sprintin sonunda *gösterilebilir ve test edilmiş* bir ürün artışı ortaya çıkar.
- Hedefe ulaşılamazsa sprint uzatılmaz; kapsam küçültülür, nedeni retrospektifte konuşulur.

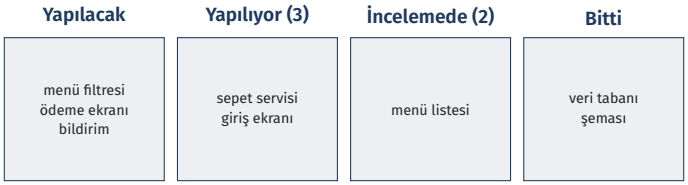
Günlük toplantı nasıl kısa kalır?

- Ayakta yapılır, 15 dakikayı geçmez, herkes sırayla üç soruyu yanıtlar.
- Sorun çözme toplantısı değildir; sorun tespit edilir, ilgililer sonra konuşur.
- Rapor verme toplantısı da değildir; ekip kendi kendine eş güdüm sağlar.
- Dönem projenizde yüz yüze zorsa, yazılı olarak da yapılabilir.

Yazılı günlük not (grup sohbetinde)

```
Dün: sepet servisini bitirdim, PR #42 açık.
Bugün: sipariş doğrulamasını yazacağım.
Engel: kantinden fiyat listesini alamadım, iletişim sorumlusu
bakabilir mi?
```

Kanban: akışı görünür kıl, yığılmayı engelle



- Parantez içindeki sayı **eş zamanlı iş sınırı** (*WIP limit*): o sütunda en çok o kadar kart olabilir.
- Sınır dolduysa yeni iş başlatmak yasaktır; tıkanıklığı açmaya gidilir.
- Sekiz kişilik grupta en sık görülen hata: on beş işin hepsinin “yapılıyor” olması.

Neden iş sınırı koyuyoruz?

- Aynı anda çok işe başlamak, daha çok iş bitirmek anlamına gelmez. Yarım kalan iş kullanıcıya sunulamaz.
- İş sınırı, ekibi yeni işe başlamak yerine mevcut işleri bitirmeye yöneltir.
- Sınır aynı zamanda darboğazı görünür kılar: kartlar hep “incelemede” birikiyorsa inceleme kapasitesi yetersizdir.

Teslim süresi

Bir kartın panoya girmesiyle “bitti” sütununa geçmesi arasındaki süredir. Kısa teslim süresi, hızlı geri bildirim demektir.

Scrum mu, Kanban mı?

- **Scrum:** sabit uzunlukta sprintler, sprint hedefi ve düzenli planlama. İş öngörülebilirse uygundur.
- **Kanban:** sürekli akış, sprint yok, sınırlar var. İş sürekli ve kesintili geliyorsa iyidir.
- Çoğu ekip ikisini birlikte kullanır: iki haftalık sprint ve pano üzerinde iş sınırı.

Bu hafta projenizde

Bu derste önerilen: iki haftalık sprint (Scrum ritmi) ve dört sütunlu pano üzerinde “yapılıyor” sütununda en çok üç kart (Kanban sınırı).

DevOps neyi çözer?

DevOps

Geliştirme ve işletmeyi tek bir sorumluluk zinciri hâline getiren çalışma biçimidir: yazan ekip, çalıştırmaktan da sorumludur.

- Eski ayrım: geliştirici kodu “duvarın üstünden atar”, işletme ayakta tutmaya çalışır.
- Sorun teknik değil, teşvik sorunudur: biri hızdan, öteki hizmetin istikrarından sorumludur.
- Çözüm: aynı ekip hem yayımlar hem sonucuna katlanır; otomasyon zorunlu hâle gelir.

Sürekli entegrasyon ve sürekli teslimat

- **Sürekli entegrasyon** (*continuous integration*): her commit sonrası derleme ve testler otomatik çalışır.
- **Sürekli teslimat** (*continuous delivery*): her başarılı derleme, yayımlanabilir bir paket üretir.
- Amaç hız değil, *geri bildirim süresini kısaltmaktır*. Hatayı on dakikada öğrenmek ile üç hafta sonra öğrenmek farklı işlerdir.
- Küçük ve sık birleştirme, uzun süren ve sorunlu birleştirmeleri azaltır.

Ölçek farkı

Büyük ürün ekiplerinde günde binlerce test otomatik koşar. Sizin projenizde on beş test yeter; önemli olan *her commit'te koşmasıdır*.

Uygulamalı örnek: ilk sprint planı

⏭ Adım 1 — Sprint hedefini tek cümleyle yaz

“Öğrenci menüyü görüp bir sipariş verebilir; işletmeci siparişi listede görür.”

⏭ Adım 2 — Hedefe hizmet eden işleri seç

Menü veri modeli, menü listeleme servisi, menü ekranı, sipariş oluşturma, işletmeci sipariş listesi. Ödeme ve bildirim bu sprintin kapsamında değildir.

Uygulamalı örnek: ilk sprint planı (devam)

⏭ Adım 3 — Kapasiteyi kontrol et

8 kişi $\times$ 2 hafta $\times$ haftada 5 saat $=$ 80 saat. Beş iş $\times$ ortalama 12 saat $=$ 60 saat. Kalan 20 saat inceleme, düzeltme ve toplantı için.

⏭ Adım 4 — Her işe tek sahip ata

Her işin tek bir sorumlusu olmalıdır; sorumlusu belirsiz veya birden çok kişi olan işler kolayca ortada kalır.

Alıştırma 2

Alıştırma (12 dk — gruplarınızla)

Aşağıdaki pano bir hafta sonunda bu hâle gelmiş. Üç sorunu bulun ve her biri için bir düzeltme önerin.

Yapılacak	Yapılıyor (3)	İncelemede (2)	Bitti
(boş)	giriş ekranı — Ali menü ekranı — Ali menü servisi — ? sipariş — Ali, Can yönetim paneli — ?	sepet servisi — ? ödeme — Ayşe bildirim — Can rapor — Ayşe	(boş)

Alıştırma 2: çözüm

Çözüm

Sorun 1 — İş sınırı ihlali. “Yapılıyor” sütununda sınır 3 ama 5 kart var; “incelemede” sınırı 2 ama 4 kart var. *Düzeltme:* yeni iş başlatmayı durdurun, incelemedeki kartları tamamlamaya öncelik verin. Darboğaz inceleme aşamasındadır.

Sorun 2 — Sorumlusu belirsiz kartlar. Üç kartın sorumlusu yok, bir kartta iki sorumlu var. *Düzeltme:* her karta tek sorumlu atayın; sorumlusu olmayan kart “yapılacak” sütununa döner.

Sorun 3 — Yük dengesizliği ve hiç biten iş olmaması. Ali dört kart taşıyor, “bitti” sütunu boş. *Düzeltme:* işleri yeniden dağıtın; bir haftada hiçbir kart bitmediyse kartlar fazla büyük, daha küçük parçalara bölün.

- Pano yalnızca bir iş listesi değildir; sürecin nerede tıkandığını da gösterir.

Bölüm 3: Git, GitHub ve depo kurulumu

Depo, dal ve commit

- **Depo** (*repository*): projenin tüm dosyaları ve tüm değişiklik geçmişi.
- **Commit** (*commit*): anlamlı tek bir değişikliğin kalıcı kaydı; mesajı *niçin* yazıldığını anlatır.
- **Dal** (*branch*): ana kod bozulmadan paralel çalışma alanı.
- **Birleştirme isteği** (*pull request*): dalı ana koda katmadan önce inceleme talebi.

⚠ Kötü commit mesajı

“güncelleme”, “düzeltme”, “son hâli” ve “asdf”. Üç ay sonra bunlar hiçbir şey anlatmaz. İyi mesaj: “sepet toplamında KDV yuvarlama hatası düzeltildi”.

Günlük iş akışı

```
git clone https://github.com/<grup>/<proje>.git
git switch -c ozellik/menu-filtresi     # yeni dal ac
# ... kod yaz, kucuk adimlarla ilerle ...
git add src/menu.py tests/test_menu.py
git commit -m "menu listesine kategori filtresi eklendi"
git push -u origin ozellik/menu-filtresi
# GitHub'da "Compare & pull request" -> inceleyici ata
git switch main && git pull             # birlesince ana dali guncelle
```

- Ana dala doğrudan yazılmaz. Her iş kendi dalında açılır ve inceleme ile girer.
- Dal adı işi anlatır: *ozellik/...*, *hata/...*, *belge/...*

Uygulamalı örnek: birleştirme çakışması

İki kişi aynı dosyanın aynı satırını değiştirdi. `git pull` şunu yazdı:

```
CONFLICT (content): Merge conflict in src/menu.py
Automatic merge failed; fix conflicts and then commit the result.
```

Dosyayı açtığınızda şunu görürsünüz:

```
<<<<<<< HEAD
fiyat = urun.fiyat * 1.20      # KDV dahil
=======
fiyat = round(urun.fiyat * 1.2, 2)
>>>>>>> ozellik/menu-filtresi
```

`HEAD` sizin dalınız, alt kısım gelen dal. Git hangisinin doğru olduğunu bilemez; kararı siz verirsiniz.

Uygulamalı örnek: çakışmayı çözmek

⏭ Adım 1 — Doğru davranışa karar ver

İkisi de KDV ekliyor; ikincisi kuruş yuvarlaması yapıyor. Doğru olan yuvarlamalı sürüm.

⏭ Adım 2 — İşaretleri sil, tek doğru satırı bırak

```
fiyat = round(urun.fiyat * KDV_ORANI, 2)  # KDV dahil, kurus yuvarlamali
```

Uygulamalı örnek: çakışmayı çözmek (devam)

⏭ Adım 3 — Testi koş, sonra tamamla

```
pytest tests/test_menu.py \&\& git add src/menu.py
git commit -m "KDV hesabinda cakisma cozuldu: yuvarlamali surum secildi"
```

Çakışmayı “ikisini de bırak” veya “rastgele birini sil” diye çözmek, en sık görülen ve en pahalı hatadır.

Çakışmayı azaltmanın yolları

- Küçük ve sık birleştirin. Bir hafta bekleyen dal, kesin çakışır.
- Dosya sahipliğini kabaca bölün: aynı dosyada iki kişi aynı gün çalışmasın.
- Ana dalı sık çekin: `git switch main && git pull` günde bir kez.
- Biçimlendirme değişikliklerini ayrı commit yapın; yoksa gerçek değişiklik kaybolur.
- `.gitignore` kullanın: derleme çıktısı, sanal ortam ve gizli dosyalar depoya girmez.

⚠ Depoya asla girmeyecekler

Parolalar, API anahtarları, `.env` dosyası, veri tabanı yedeği, öğrenci kişisel verisi. Bir kez girdiyse geçmişten silmek zordur; anahtarı iptal etmek gerekir.

Kod incelemenin amacı

- Hata bulmak yalnızca bir yararıdır; asıl yarar *bilgi yaymaktır*.
- İnceleme kişiyi değil değişikliği değerlendirir. Dil buna göre kurulur.
- Küçük PR hızlı incelenir. Bin satırlık PR incelenmez, onaylanır.
- Bu derste kural: her birleştirme isteği en az bir kişi tarafından incelenir.

İyi inceleme yorumu

“Burada kullanıcı girdisi doğrudan sorguya gidiyor; parametreli sorgu kullanalım mı? Hafta 9’daki enjeksiyon örneğine benziyor.” Somut, gerekçeli, saldırısız.

Grup deposu nasıl kurulur?

1. Bir GitHub kuruluşu açın; adı *bz313-2026-grupNN* olsun.
2. Depoyu oluşturun; tüm üyeleri yazma yetkisiyle ekleyin; öğretim üyesini de ekleyin.
3. *main* dalını korumaya alın: doğrudan gönderim kapalı, en az bir inceleme zorunlu.
4. *README.md* yazın: proje adı, müşteri, üyeler, kurulum adımları.
5. *docs/* klasörü açın: ödev raporları ve görüşme tutanakları buraya gider.
6. *.gitignore* ve *LICENSE* dosyalarını ekleyin.
7. Projects sekmesinden dört sütunlu pano kurun ve iş sınırlarını yazın.

Panoyu iş birikimiyle beslemek

- Her iş için bir *iş kaydı* (issue) açılır: başlık beklenen sonucu, açıklama ise kabul ölçütlerini belirtir.
- İş kayıtları panoda karta dönüşür; her kartın tek sorumlusu vardır.
- Birleştirme isteği açıklamasına `Closes #42` yazarsanız iş kendiliğinden kapanır.
- Kilometre taşları, ödev teslimleriyle eşlenir: Ödev 1, Ödev 2, Ödev 3, Ödev 4.

İyi bir iş kaydı

Başlık: “Menü listesi kategoriye göre filtrelenebilsin”.
Kabul ölçütü: “Kullanıcı kategori seçince yalnızca o kategorinin ürünleri listelenir; seçim kaldırılınca tüm liste döner.”

Kullandığınız her kaynağın bir sahibi vardır

- Hazır kütüphane, kod parçası, ikon, yazı tipi, veri kümesi: hepsi lisanslıdır.
- Lisansı okumadan kullanmak, ürününüzü hukuken riskli hâle getirir.
- Üç yaygın aile: izin verici (MIT, Apache-2.0), koşullu paylaşım (GPL), tescilli.
- Yapay zekâ ile üretilen kodu da belirtin; kod nasıl üretilmiş olursa olsun sorumluluk sizdedir.

Bu hafta projenizde

Deponuza *LICENSE* dosyası ekleyin ve *README* içinde kullandığınız başlıca kütüphaneleri lisanslarıyla listeleyin. Bu konu Ödev 4'te değerlendirilecektir.

Bir iş ne zaman biter? Bitti tanımı

Bitti tanımı

Bir işin “bitti” sütununa geçebilmesi için sağlaması gereken, ekibin önceden üzerinde anlaştığı koşullar listesidir.

- Kod yazıldı ve ana dala birleştirildi.
- En az bir kişi inceledi ve onayladı.
- Testleri var ve tüm testler geçiyor.
- Kabul ölçütleri iş kaydında yazılıdır ve karşılanmıştır.
- Gerekiyorsa *README* veya belge güncellendi.

Bu liste olmadan “bitti” kişiye göre değişir ve pano yalan söyler.

Dal stratejisi: iki yaygın seçenek

	Ana dal odaklı	Sürüm dalı odaklı
Nasıl	Kısa ömürlü dallar, günde birleşme	Uzun ömürlü geliştirme ve sürüm dalları
Uygun	Sık yayım yapan ekipler	Aynı anda birden çok sürümü destekleyen ürünler
Zorluk	Ana dal her an çalışır olmalı	Dallar arası aktarma yükü

Bu hafta projenizde

Dönem projeniz için *ana dal odaklı* strateji önerilir: `main` korumalı, her iş için kısa ömürlü bir dal, iki gün içinde birleştirme. Karmaşık dal ağacı sekiz kişilik grupta kaybolmanıza yol açar.

Sürüm numaralandırma

- Biçim: *ANA.ALT.YAMA* — örneğin `1.4.2`.
- **ANA** artar: geriye dönük uyumu bozan değişiklik.
- **ALT** artar: uyumu bozmayan yeni yetenek.
- **YAMA** artar: yalnızca hata düzeltmesi.
- Etiketleme: `git tag -a v1.0.0 -m "musteri kabul surumu"`

Neden işinize yarar?

Müşteriye “şu an hangi sürümü kullanıyorsunuz?” diye sorabilmek, hatayı yeniden üretmenin ilk adımıdır. Sürümsüz sistemde bu soru yanıtsız kalır.

Sürekli entegrasyonu on satırda kurmak

Deponuzda *.github/workflows/test.yml* dosyası:

```
name: testler
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: pytest -q
```

Sürekli entegrasyonu on satırda kurmak (devam)

- Her gönderim ve her birleştirme isteğinde testler kendiliğinden koşar.
- Bu derste zorunlu değildir; kuran gruplar hata bulmakta belirgin biçimde öne geçer.

İş kaydı şablonu: ekibin ortak dili

Şablon

Başlık: Ne olmasını istiyoruz? (sonuç cümlesi)
Neden: Hangi kullanıcı ihtiyacına hizmet ediyor?
Kabul ölçütü: Hangi koşullar sağlanınca bitmiş sayılır?
Notlar: Ekran taslağı, ilgili iş kaydı, teknik kısıt.
Etiketler: öncelik, alan (arayüz, servis, veri), tahmin

- Şablon, üç aylık projede “bu neydi?” sorusunu ortadan kaldırır.
- GitHub’da `.github/ISSUE_TEMPLATE/` klasörüne konur ve otomatik açılır.

Alıştırma 3

Alıştırma (10 dk — ikili çalışın)

Aşağıdaki depo geçmişinde beş sorun var. Her birini bulun ve düzeltmesini yazın.

```
* 8f2a1c (main) son hali
* 3d9b7e asdf
* c14e02 duzeltme
* 91ab55 Merge branch 'main' into main
* 77cd31 ozellik + hata duzeltmesi + css duzenleme (842 satir)
* 05ef19 .env dosyasi eklendi
```

Not: tüm commit'ler doğrudan *main* dalına yapılmış; hiç birleştirme isteği açılmamış.

Alıştırma 3: çözüm

Çözüm

- **Anlamsız mesajlar** (“son hali”, “asdf”, “duzeltme”). *Düzeltme:* mesaj neyin niçin değiştiğini söyler: “sepet toplamında KDV yuvarlama hatası düzeltildi”.
- **Gereksiz birleştirme kaydı** (`Merge branch 'main' into main`). *Düzeltme:* çalışmaya başlamadan `git pull` yapın; gereksiz birleştirme commit’i oluşmaz.
- **Çok işi tek commit’te toplamak** (842 satır, üç ayrı iş). *Düzeltme:* her iş kendi commit’inde; incelenebilir boyutta tutun.
- **Gizli bilgi deposunda** (`.env` dosyası). *Düzeltme:* `.gitignore` ile dışla, `.env.example` ekle, sızan anahtarları iptal et.
- **Doğrudan `main` dalına yazmak.** *Düzeltme:* `main` dalını korumaya alın; her iş kendi dalında açılıp en az bir inceleme ile girsin.

Çıkış bileti: kendi cümlelerinizle

1. Rezervasyon için veri, servis ve ekran içeren tek bir dikey dilim yazın.
2. Kod yazıldı ama incelenmedi: kart neden henüz bitmiş sayılmayabilir?
3. Panonuzdaki bir tıkanmayı görünür kılacak kural önerin.

Yanıt biçimi

Her soruya kısa bir yanıt yazın; kararınızı bir gerekçeyle destekleyin. Ardından yanınızdaki kişiyle karşılaştırın.

Bu hafta projenizde

Bu hafta projenizde

- GitHub kuruluşunu ve depoyu kurun; tüm üyeleri ve öğretim üyesini ekleyin.
- `main` dalını korumaya alın; en az bir inceleme zorunlu olsun.
- `README.md`, `docs/`, `.gitignore` ve `LICENSE` ekleyin.
- Dört sütunlu Kanban panosunu kurun; “yapılıyor” sınırını üç yapın.
- Grup listenizi ve müşteri onay formunuzu teslim edin.
- İlk sprintinizin hedefini tek cümleyle yazın ve başlangıç tarihini belirleyin.

Kaynaklar

- Çevik Yazılım Geliştirme Manifestosu (2001). https://agilemanifesto.org/iso/tr/manifesto.html
- Schwaber, K. ve Sutherland, J. (2020). *Scrum Kılavuzu*. https://scrumguides.org
- Forsgren, N., Humble, J. ve Kim, G. (2018). *Accelerate*. IT Revolution.
- DORA. *State of DevOps Report*. https://dora.dev
- Chacon, S. ve Straub, B. *Pro Git*, 2. baskı. https://git-scm.com/book/tr/v2
- GitHub Docs. *Projects* ve *Protected branches*.
- Google. *Engineering Practices: Code Review Guidelines*.
- Semantic Versioning 2.0.0. https://semver.org

Gelecek hafta

Fizibilite çalışması ve müşteri görüşme teknikleri.

Ödev 1 verilecek: Fizibilite Raporu.